

로깅

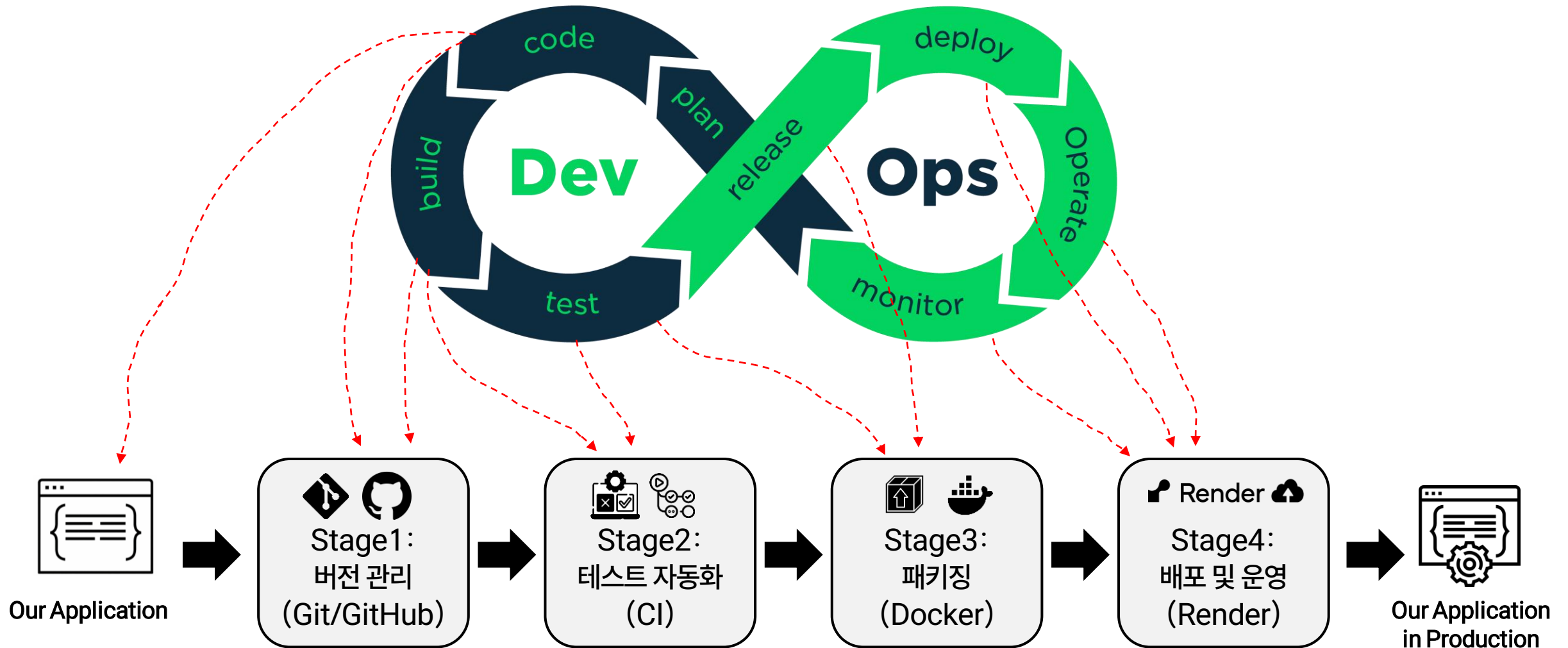
김미수

전남대학교 AI융합대학 인공지능학부

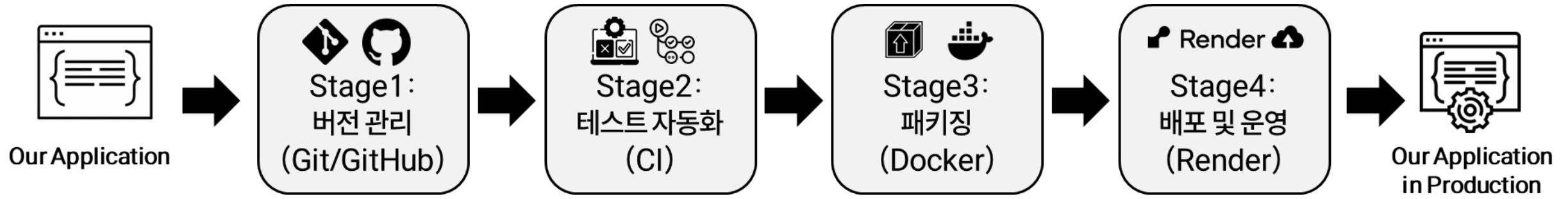
misoo.kim@jnu.ac.kr

<https://sites.google.com/view/semi-jnu>

수업에서 진행할 파이프라인



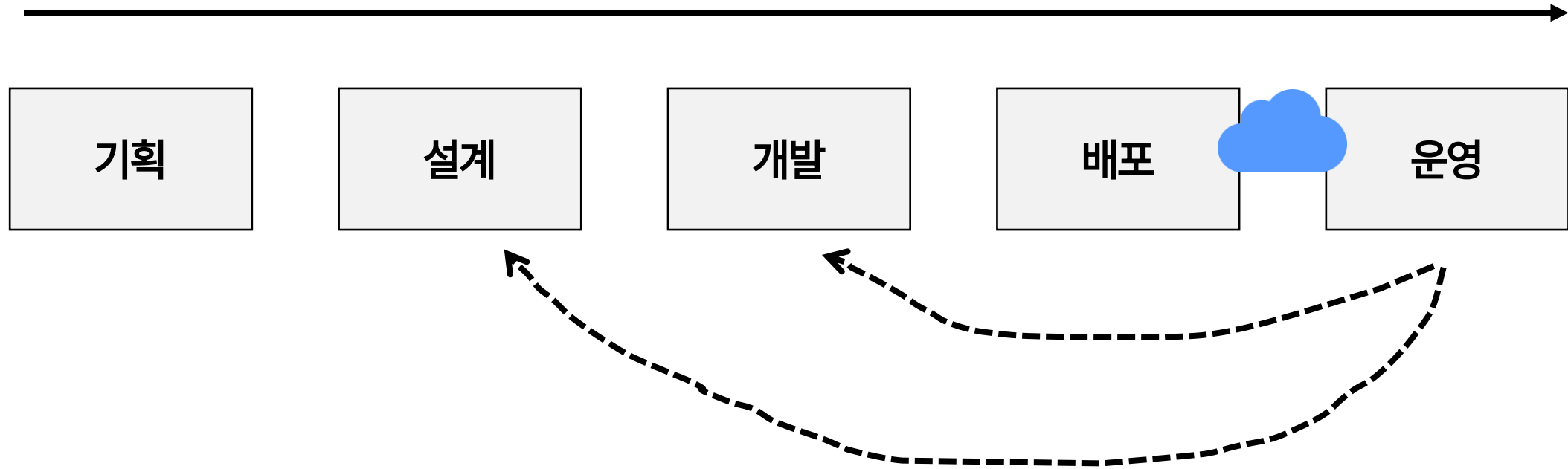
DevOps 파이프라인



- 3단계: 배포와 운영 – 서비스하고 살아있는지/문제없는지 지켜보기

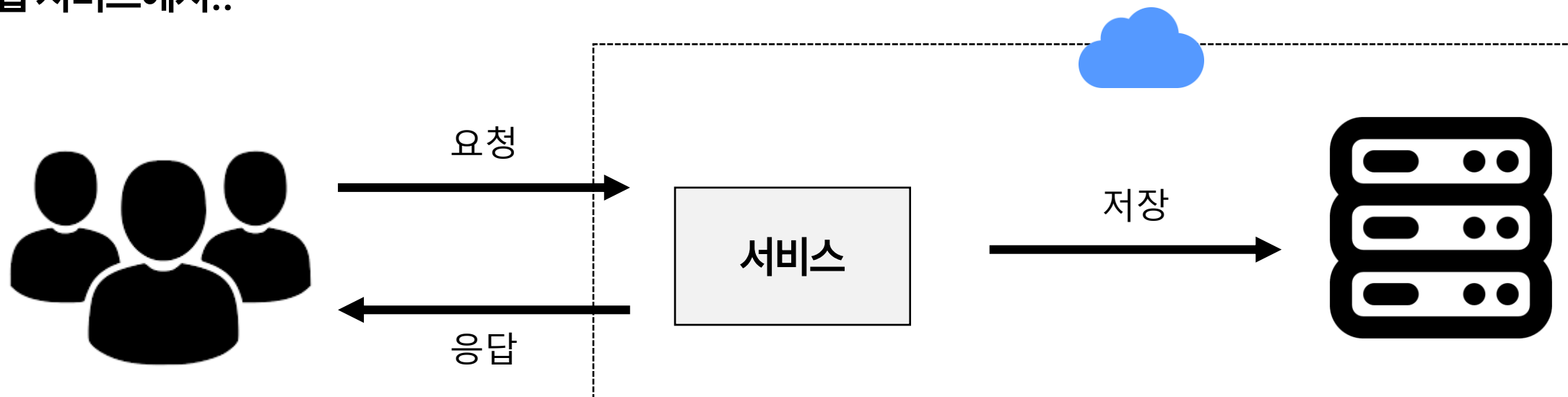
- 배포: 컨테이너화된 애플리케이션을 배포하여 누구나 접근할 수 있게 하는 작업
- 모니터링: 애플리케이션/운영 로그를 분석하여 문제 확인 후 디버깅
- 사용 도구: Render - 최소한의 배포 환경으로 간단함
 - GCP + IaC (Infrastructure as Code) : 복잡함, 수업 X

SW 개발 및 운영 사이클



운영 중 문제 확인 방법

- 웹 서비스에서..



```
9/1/99, 10:46:11, 1570, 509, 5297, 200, 0, GET, /cfdocs/akonline/paintbrush.JPG, -,
9/1/99, 10:46:49, 37703, 577, 24402, 200, 0, GET, /cfdocs/akonline/email_book.cfm,
tfirstname=francis&lastname=snitt&tid=2706PASSWORD=teachme6USERNAME=francis,
9/1/99, 10:49:11, 181500, 579, 114931, 200, 0, GET, /cfdocs/akonline/update_table.cfm,
tfirstname=francis&lastname=snitt&tid=2706PASSWORD=teachme6USERNAME=francis,
9/1/99, 10:52:04, 354641, 662, 163301, 200, 0, GET, /cfdocs/AR00LINE/assess/PAT11B.pdf,
tfirstname=francis&lastname=snitt&tid=2706PASSWORD=teachme6USERNAME=francis,
9/1/99, 10:52:31, 20921, 609, 163301, 200, 0, GET, /cfdocs/AR00LINE/assess/PAT11B.pdf,
tfirstname=francis&lastname=snitt&tid=2706PASSWORD=teachme6USERNAME=francis,
9/1/99, 10:55:30, 178985, 658, 4374, 200, 0, GET, /cfdocs/akonline/adobe.get.cfm,
tfirstname=francis&lastname=snitt&tid=2706PASSWORD=teachme6USERNAME=francis,
9/1/99, 10:55:50, 8437, 583, 39420, 200, 0, GET, /cfdocs/akonline/image.JPG, -,
9/1/99, 11:00:25, 5422, 662, 172695, 200, 0, GET, /cfdocs/AR00LINE/assess/TBT11B.pdf,
tfirstname=francis&lastname=snitt&tid=2706PASSWORD=teachme6USERNAME=francis,
9/1/99, 11:03:17, 171359, 437, 172695, 200, 0, GET, /cfdocs/AR00LINE/assess/TBT11B.pdf,
tfirstname=francis&lastname=snitt&tid=2706PASSWORD=teachme6USERNAME=francis,
9/1/99, 11:40:46, 449531, 582, 1441, 200, 0, GET, /cfdocs/akonline/chooseTableMenu.cfm,
tfirstname=francis&lastname=snitt&tid=2706PASSWORD=teachme6USERNAME=francis,
9/1/99, 11:41:16, 812, 775, 438, 200, 0, POST, /cfdocs/akonline/exampleTable.cfm,
tfirstname=francis&lastname=snitt&tid=2706PASSWORD=teachme6USERNAME=francis
```

로그란

- **프로그램의 실행 중 발생하는 정보(이벤트)를 기록한 것**
 - 주로 시스템 상태, 오류, 사용자 활동, 흐름 추적 등을 담음
- **왜 로그가 필요한가?**
 - 운영 중 문제 분석 및 해결
 - 개인 정보를 다루는 서비스나 금융 서비스는 법적으로 의무화
 - 최근 AI의 신뢰성 확보를 위해 AI 응답의 로그화가 의무화가 되고 있음 (책임 소지를 위해)

[illegible]

```

9/1/99, 18:46:10, 1578, 509, 5207, 200, 0, GET, /cfdocs/akonline/paintbrush.JPG, -,
9/1/99, 18:46:49, 27783, 501, 2482, 200, 0, GET, /cfdocs/akonline/email_book.cfm,
cfdocs/akonline/francislattane-smlttit6d270b955UUD-teachme6USERWNIW=francis,
9/1/99, 18:46:11, 184580, 579, 114931, 200, 0, GET, /cfdocs/akonline/update_table.cfm,
francislattane-francislattane-smlttit6d270b955UUD-teachme6USERWNIW=francis,
9/1/99, 18:52:08, 354641, 662, 163381, 200, 64, GET, /cfdocs/AKOHLN/assess/PAT118.pdf,
francislattane-francislattane-smlttit6d270b955UUD-teachme6USERWNIW=francis,
9/1/99, 18:52:31, 20921, 609, 163381, 200, 0, GET, /cfdocs/AKOHLN/assess/PAT118.pdf,
francislattane-francislattane-smlttit6d270b955UUD-teachme6USERWNIW=francis,
9/1/99, 18:55:38, 178985, 658, 4314, 200, 0, GET, /cfdocs/akonline/adobe get.cfm,
francislattane-francislattane-smlttit6d270b955UUD-teachme6USERWNIW=francis,
9/1/99, 18:55:58, 8437, 583, 39428, 200, 0, GET, /cfdocs/akonline/image.JPG, -,
9/1/99, 19:20:25, 5422, 662, 172695, 200, 0, GET, /cfdocs/AKOHLN/assess/10T118.pdf,
francislattane-francislattane-smlttit6d270b955UUD-teachme6USERWNIW=francis,
9/1/99, 19:38:17, 171359, 437, 172695, 200, 0, GET, /cfdocs/AKOHLN/assess/10T118.pdf,
francislattane-francislattane-smlttit6d270b955UUD-teachme6USERWNIW=francis,
9/1/99, 19:40:46, 449521, 582, 1441, 200, 0, GET, /cfdocs/akonline/chooseTableMenu.cfm,
francislattane-francislattane-smlttit6d270b955UUD-teachme6USERWNIW=francis,
9/1/99, 19:41:16, 812, 775, 438, 200, 0, POST, /cfdocs/akonline/exampleTable.cfm,
francislattane-francislattane-smlttit6d270b955UUD-teachme6USERWNIW=francis,

```

```

C:\Windows\system32\cmd.exe /c dir C:\ProgramData\Apache24\logs\*.* /b /s /a:dcn /q | findstr /i /c:"*.log" | sort /r
LogFile Name,LogPath,RemoteHostName,RemoteLogName,UserName,Date,Time,Request,Status,Bytes,Sent,Referer,User-Agent,Cookie
C:\ProgramData\Apache24\logs\access_log.25 : 1, 2017-07-12 04:57:49, GET /mysql/test2.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.26 : 1, 2017-07-12 04:57:51, GET /mysql/test2.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.27 : 1, 2017-07-12 04:58:03, GET /mysql/test2.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.29 : 3, 2017-07-12 04:58:31, GET /mysql/test2.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.30 : 1, 2017-07-12 02:10:05, GET /phpMyAdmin/ HTTP/1.1, 500, 1019, ...
C:\ProgramData\Apache24\logs\access_log.34 : 1, 2017-07-13 02:10:25, GET /phpMyAdmin/ HTTP/1.1, 500, 1078, ...
C:\ProgramData\Apache24\logs\access_log.35 : 1, 2017-07-13 02:10:27, GET /phpMyAdmin/ HTTP/1.1, 500, 1078, ...
C:\ProgramData\Apache24\logs\access_log.38 : 1, 2017-07-13 02:12:56, GET /phpMyAdmin/ HTTP/1.1, 500, 1078, ...
C:\ProgramData\Apache24\logs\access_log.57 : 1, root, 2017-07-13 02:20:45, GET /phpMyAdmin/ HTTP/1.1, 500, 1019, ...
C:\ProgramData\Apache24\logs\access_log.59 : 1, root, 2017-07-13 02:20:52, GET /phpMyAdmin/ HTTP/1.1, 500, 1019, ...
C:\ProgramData\Apache24\logs\access_log.60 : 1, root, 2017-07-13 02:20:59, GET /phpMyAdmin/ HTTP/1.1, 500, 1019, ...
C:\ProgramData\Apache24\logs\access_log.117 : 1, 2017-11-19 08:00:03, GET /test1.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.1197 : 1, 2017-11-19 02:43:02, GET /test1.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.1337 : 1, 2017-11-19 06:03:29, GET /test1.php HTTP/1.1, 500, 51, ...
C:\ProgramData\Apache24\logs\access_log.1349 : 1, 2017-11-19 06:07:14, GET /test1.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.1350 : 1, 2017-11-19 06:07:15, GET /test1.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.1351 : 1, 2017-11-19 06:07:16, GET /test1.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.1352 : 1, 2017-11-19 06:07:16, GET /test1.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.1353 : 1, 2017-11-19 06:07:16, GET /test1.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.1354 : 1, 2017-11-19 06:07:17, GET /test1.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.1355 : 1, 2017-11-19 06:07:17, GET /test1.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.1356 : 1, 2017-11-19 06:07:17, GET /test1.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.1357 : 1, 2017-11-19 06:07:17, GET /test1.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.1358 : 1, 2017-11-19 06:07:21, GET /test1.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.1359 : 1, 2017-11-19 06:07:21, GET /test1.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.1360 : 1, 2017-11-19 06:07:21, GET /test1.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.1361 : 1, 2017-11-19 06:07:21, GET /test1.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.1366 : 1, 2017-11-19 06:09:02, GET /test1.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.1367 : 1, 2017-11-19 06:09:03, GET /test1.php HTTP/1.1, 500, ...
C:\ProgramData\Apache24\logs\access_log.1368 : 1, 2017-11-19 06:09:08, GET /test1.php HTTP/1.1, 500, ...

```

로그로 남겨야 하는 것들

- 모든 것들 답아야 하나? ► 용량만 크고 문제 해결을 오히려 어렵게 함

- 문제를 분석하고 해결하기 위한 정보들 (+법적인 것들)

- | | |
|---------------|---|
| 1. 요청/응답 로그 | : 사용자가 어떤 API를 호출했고 결과는 어떤가? |
| 2. 오류 및 예외 로그 | : 시스템에서 발생한 문제의 정확한 원인을 추적 (사용자에게는 숨기기) |
| 3. 사용자 활동 로그 | : 어떤 기능이 자주 사용? 악의적인 활동 없나? 어떤 기능에서 문제? |
| 4. 시스템 상태 로그 | : 시스템 자원 상태(CPU/메모리 과부하, 디스크 부족 등) 문제 없나? |
| 5. DB 쿼리 로그 | : SQL 쿼리 기록. 느린 쿼리는 없는가? 데이터 접근 시도가 있었나? |

등등... 기타 디버깅을 위한 것들

예외 vs 로그

- 예외: 프로그램이 감지하고 처리 ► 문제가 발생했음을 감지하는 메커니즘
- 로그: 문제를 기록해서 나중에 원인을 분석하게 해주는 수단
- 예외 발생 → 로그로 기록 → 문제 원인 추적 *or* 로그로 기록 → 향후 문제 확인 → 문제 원인 추적
 - 운영 환경에서는 로그만 보고 문제를 추적해야 하므로, 주로 “모든 예외를 항상 로그로 남김”

```
try:
    num = int(input("숫자를 입력하세요: "))
    result = 10 / num
except ZeroDivisionError:
    print("0으로 나눌 수 없습니다.")
```

```
import logging
logging.basicConfig(level=logging.INFO)

try:
    num = int(input("숫자 입력: "))
    result = 10 / num
    logging.info(f"나눗셈 결과: {result}")
except ZeroDivisionError:
    logging.error("0으로 나눴습니다!") # 예외 정보를 로그로 남김
```

```
import logging

def divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        logging.exception("나눗셈 오류 발생") # 예외 정보와 함께 로그에 남김
        return None
```


로그의 기본 구성

- 시간(timestamp) : 로그 기록 시간과 날짜
- 로그 레벨 : 로그 중요도 수준. 중요도 기준으로 필터링 가능
- 위치 정보 : 로그를 기록한 파일/클래스/함수명 및 라인 번호 등
- 메시지 : 기록하고 싶은 내용 및 상태 정보 (개발자 설정)
- 환경 정보 : PID, 스레드 이름 등등..

(개발자와 로그 수집/기록 도구마다 상이할 수 있음)

```
PID
2024-11-29T00:10:30.354+09:00 INFO 9736 --- [nio-8080-exec-2] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializ
2024-11-29T00:10:30.354+09:00 INFO 9736 --- [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet : Initializ
2024-11-29T00:10:30.355+09:00 INFO 9736 --- [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet : Completed
2024-11-29T00:10:30.521+09:00 INFO 9736 --- [nio-8080-exec-2] k.c.s.a.SimpleShortenUrlService : shortenUr
2024-11-29T00:11:26.845+09:00 ERROR 9736 --- [nio-8080-exec-3] k.c.s.p.GlobalExceptionHandler : 단축 URL을
```

시간

레벨

스레드 이름

패키지 + 클래스

로그 레벨

- 왜 필요한가?: 어떤 메시지를 기록할지 필터링 하는 기준, 너무 많은 로그는 성능 저하 및 가독성 문제 유발

- 종류

1. TRACE : 코드의 세부적인 실행 경로까지 포함

2. DEBUG : 개발/디버깅용 상세 정보

3. INFO : 일반적인 실행 정보

4. WARN : 문제 가능성이 있는 정보

5. ERROR : 오류 발생

6. FATAL : 치명적인 오류 발생 (or CRITICAL)

		Detail Included in Logs				
		Debug statements	Informational messages	Warning statements	Error statements	Fatal statements
Logging Level	ALL	Included	Included	Included	Included	Included
	DEBUG	Included	Included	Included	Included	Included
	INFO	Excluded	Included	Included	Included	Included
	WARN	Excluded	Excluded	Included	Included	Included
	ERROR	Excluded	Excluded	Excluded	Included	Included
	FATAL	Excluded	Excluded	Excluded	Excluded	Included
	OFF	Excluded	Excluded	Excluded	Excluded	Excluded

		Event Level					
		TRACE	DEBUG	INFO	WARN	ERROR	FATAL
Config Level	ALL	o	o	o	o	o	o
	TRACE	o	o	o	o	o	o
	DEBUG	X	o	o	o	o	o
	INFO	X	X	o	o	o	o
	WARN	X	X	X	o	o	o
	ERROR	X	X	X	X	o	o
	FATAL	X	X	X	X	X	o
	OFF	X	X	X	X	X	X

로그 레벨

- **DEBUG** : 개발/디버깅용 상세 정보

- 내부 로직, 흐름, 변수 상태 등 개발 중에만 필요한 정보만 기록하는 수준

상황	로그 메시지
로그인 함수 진입	DEBUG - login(userId=1234) 함수 호출됨
반복문 내부 변수 상태	DEBUG - i=3, current_total=47
요청 파라미터 확인	DEBUG - POST /order - params: {item_id=123, qty=2}
API 응답 구조 확인	DEBUG - 응답 데이터: {'status': 'ok', 'count': 10}

- **INFO** : 일반적인 실행 정보

- 정상적인 작업 흐름 기록. 개발자가 생각하기에 기록할 만한 이벤트들

상황	로그 메시지
로그인 성공	INFO - userId=1234 로그인 성공
게시글 등록 완료	INFO - postId=784 등록 완료 by userId=3002
서비스 시작	INFO - 서버 시작: 0.0.0.0:8080
배치 작업 완료	INFO - 일일 보고서 생성 완료 (rows=1342)

로그 레벨

- **WARN** : 문제 가능성이 있는 정보

- 시스템에 큰 문제는 아니지만, 주의가 필요한 상황

상황	로그 메시지
디스크 사용량 과다	WARNING - 디스크 사용량 90% 초과 (/var/log)
설정 누락 → 기본값 사용	WARNING - config.yml 없음. 기본 설정 적용됨
타임아웃 → 재시도	WARNING - 외부 API 응답 지연. 1초 후 재시도
오래된 브라우저 접근	WARNING - 사용자가 지원되지 않는 브라우저 사용: IE11

- **ERROR** : 오류 발생

- 예외 발생 등으로 인해 기능이 실패한 상황

상황	로그 메시지
DB 쿼리 실패	ERROR - 유저 조회 실패: userId=9999 존재하지 않음
결제 실패	ERROR - 결제 실패. 카드 한도 초과
파일 저장 실패	ERROR - 첨부파일 저장 실패: PermissionDenied
외부 API 호출 예외	ERROR - Kakao API 호출 실패 (status=503)

로그 레벨

- FATAL (CRITICAL) : 시스템이 멈출 수 있는 심각한 장애 상황. 치명적인 오류

상황	로그 메시지
DB 연결 끊김	CRITICAL - PostgreSQL 연결 실패. 서비스 중단됨
캐시 서버 다운	CRITICAL - Redis down. 모든 API 지연 발생 중
인증서 만료	CRITICAL - SSL 인증서 만료. 서비스 접근 불가
시스템 종료 전 메시지	CRITICAL - 시스템 shutdown 감지됨. 긴급 로그 저장 중

▶ 나누면 할 수 있는 것들: 적절한 검토 등 운영 중 문제 상황에 신속한 대처 전략 수립 가능

▶ 로깅 작업을 구현하면서 발생 가능한 에러를 더 꼼꼼히 검토할 수 있음!

<전략 예시>



좋은 로그 vs 나쁜 로그

- 좋은 로그란?

- 문제가 발생한 맥락과 의미를 명확히 전달하고, 검색/분석/추적이 용이한 형식으로 구성된 로그

상황	좋은 메시지
로그인 성공	✓ INFO - 로그인 성공: user_id=1234, ip=192.168.0.5
결제 오류	✓ ERROR - 결제 실패: user_id=1234, 카드 한도 초과
함수 진입	✓ DEBUG - start process_order(order_id=abc123)
API 응답 실패	✓ WARNING - KakaoPay 응답 지연 (request_id=xyz99, retry=1/3)
파일 저장 실패	✓ ERROR - 첨부파일 저장 실패: path=/uploads/abc.png, reason=PermissionDenied

상황	나쁜 메시지	문제점
로그인 실패	✗ "로그인 실패"	누가 실패했는지, 이유가 없음
파일 저장 에러	✗ "에러 발생!!"	어떤 에러인지 알 수 없음
API 호출 문제	✗ "안 됨"	언제? 어떤 API? 어느 함수에서? 아무 정보 없음
예외 처리	✗ print(e)	콘솔에는 나오지만, 로그로 남지 않음
경고 메시지	✗ "주의"	이유도 맥락도 없음

좋은 로그를 작성하기 위한 로그 템플릿

- 시간(timestamp) : 로그 기록 시간과 날짜
- 로그 레벨 : 로그 중요도 수준. 중요도 기준으로 필터링 가능
- 위치 정보 : 로그를 기록한 파일/클래스/함수명 및 라인 번호 등
- 메시지 : 기록하고 싶은 내용 및 상태 정보 (개발자 설정)
- 환경 정보 : PID, 스레드 이름 등등..

(개발자와 로그 수집/기록 도구마다 상이할 수 있음)

```
PID
2024-11-29T00:10:30.354+09:00 INFO 9736 --- [nio-8080-exec-2] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializ
2024-11-29T00:10:30.354+09:00 INFO 9736 --- [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet : Initializ
2024-11-29T00:10:30.355+09:00 INFO 9736 --- [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet : Completed
2024-11-29T00:10:30.521+09:00 INFO 9736 --- [nio-8080-exec-2] k.c.s.a.SimpleShortenUrlService : shortenUr
2024-11-29T00:11:26.845+09:00 ERROR 9736 --- [nio-8080-exec-3] k.c.s.p.GlobalExceptionHandler : 단축 URL을
```

시간

레벨

스레드 이름

패키지 + 클래스

좋은 로그를 작성하기 위한 로그 템플릿

- 직접 설정

종류	템플릿	출력 예시
기본	'[% (asctime)s] %(levelname)s - %(message)s'	[2025-05-19 13:42:12] INFO - 로그인 성공: user_id=abc123
상세	'[% (asctime)s] %(levelname)s [% (name)s.%(funcName)s: %(lineno)d] - %(message)s'	[2025-05-19 13:42:12] ERROR [OrderService.process_order:74] - 결제 실패: 주문 ID=ord123, 이유=카드 한도 초과
구조화 (json)	'{"time": "%(asctime)s", "level": "%(levelname)s", "module": "%(module)s", "function": "%(funcName)s", "line": %(lineno)d, "message": "%(message)s"}'	{ "time": "2025-05-19 13:42:12", "level": "WARNING", "module": "inventory", "function": "update_stock", "line": 45, "message": "재고 수량 부족: item_id=itm4321, 요청=10, 재고=2" }

- 실무: 도구를 사용해 데이터만 전달

로깅 방법

- 쉬운 도구: python logging 라이브러리

```
import logging

# 로그 생성
logger = logging.getLogger()

# 로그의 출력 기준 설정
logger.setLevel(logging.INFO)

# log 출력 형식
formatter = logging.Formatter("%(asctime)s - %(name)s - %(levelname)s - %(message)s")

# log를 console에 출력
stream_handler = logging.StreamHandler()
stream_handler.setFormatter(formatter)
logger.addHandler(stream_handler)

# log를 파일에 출력
file_handler = logging.FileHandler("my.log")
file_handler.setFormatter(formatter)
logger.addHandler(file_handler)

for i in range(10):
    logger.info(f"{i}번째 방문입니다.")
```

로깅 방법

- 쉬운 도구: python logging 라이브러리
 - getLogger(__name__)
 - 이름 안주면 root logger
 - __name__: 로거 이름이 모듈명으로 설정
 - ▶ 싱글턴 패턴이 적용되어 있음. 단일 객체.

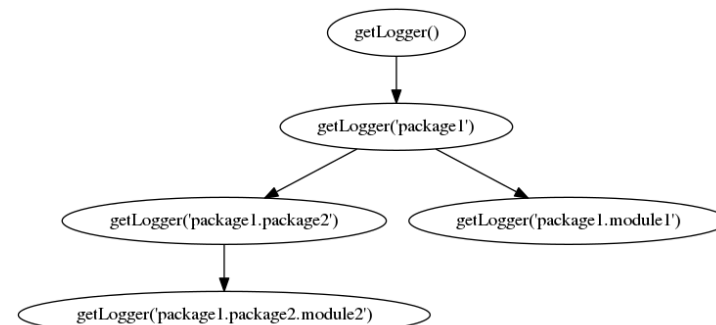
```
# myapp.py
import logging
import mylib
logger = logging.getLogger(__name__)

def main():
    logging.basicConfig(filename='myapp.log', level=logging.INFO)
    logger.info('Started')
    mylib.do_something()
    logger.info('Finished')

if __name__ == '__main__':
    main()
```

```
# myLib.py
import logging
logger = logging.getLogger(__name__)

def do_something():
    logger.info('Doing something')
```



```
import logging

# 로그 생성
logger = logging.getLogger()

# 로그의 출력 기준 설정
logger.setLevel(logging.INFO)

#log 출력 형식
formatter = logging.Formatter("%(asctime)s - %(name)s - %(levelname)s - %(messages)s")

# log를 console에 출력
stream_handler = logging.StreamHandler()
stream_handler.setFormatter(formatter)
logger.addHandler(stream_handler)

#log를 파일에 출력
file_handler = logging.FileHandler("my.log")
file_handler.setFormatter(formatter)
logger.addHandler(file_handler)

for i in range(10):
    logger.info(f"{i}번째 방문입니다.")
```

로깅 방법

- 쉬운 도구: python logging 라이브러리
 - setLevel(logging.TYPE)
 - 설정한 타입의 로그만 기록

Level	Value	When to use
DEBUG	10	자세한 정보
INFO	20	정상 정보
WARNING	30	주의할 이슈
ERROR	40	문제
CRITICAL	50	심각한 문제

```
CRITICAL = 50
FATAL = CRITICAL
ERROR = 40
WARNING = 30
WARN = WARNING
INFO = 20
DEBUG = 10
NOTSET = 0
```

```
if level > 세팅된 레벨:
    출력
```

```
if ERROR > INFO :   세팅이 INFO 면 ERROR 는 출력
if DEBUG > INFO :   세팅이 INFO 면 DEBUG 는 출력하지 않음
```

```
import logging

# 로그 생성
logger = logging.getLogger()

# 로그의 출력 기준 설정
logger.setLevel(logging.INFO)

#log 출력 형식
formatter = logging.Formatter("%(asctime)s - %(name)s - %(levelname)s - %(messages)s")

# log를 console에 출력
stream_handler = logging.StreamHandler()
stream_handler.setFormatter(formatter)
logger.addHandler(stream_handler)

#log를 파일에 출력
file_handler = logging.FileHandler("my.log")
file_handler.setFormatter(formatter)
logger.addHandler(file_handler)

for i in range(10):
    logger.info(f"{i}번째 방문입니다.")
```

로깅 방법

- 쉬운 도구: python logging 라이브러리

- Format

- <https://docs.python.org/3/library/logging.html#logrecord-attributes>

- 파일 별로 Logger에 붙인 name → logger내 name 필드에 저장 → 특정 파일에서 logging 시 자동 설정

asctime	%(asctime)s	Human-readable time when the <code>LogRecord</code> was created. By default this is of the form '2003-07-08 16:49:45,896' (the numbers after the comma are millisecond portion of the time).
created	%(created)f	Time when the <code>LogRecord</code> was created (as returned by <code>time.time_ns()</code> / <code>1e9</code>).
exc_info		You shouldn't need to format this yourself. Exception tuple (a la <code>sys.exc_info</code>) or, if no exception has occurred, <code>None</code> .
filename	%(filename)s	Filename portion of pathname.
funcName	%(funcName)s	Name of function containing the logging call.
levelname	%(levelname)s	Text logging level for the message ('DEBUG', 'INFO', 'WARNING', 'ERROR', 'CRITICAL').
levelno	%(levelno)s	Numeric logging level for the message (<code>DEBUG</code> , <code>INFO</code> , <code>WARNING</code> , <code>ERROR</code> , <code>CRITICAL</code>).
lineno	%(lineno)d	Source line number where the logging call was issued (if available).
message	%(message)s	The logged message, computed as <code>msg % args</code> . This is set when <code>Formatter.format()</code> is invoked.
module	%(module)s	Module (name portion of <code>filename</code>).
msecs	%(msecs)d	Millisecond portion of the time when the <code>LogRecord</code> was created.
msg		You shouldn't need to format this yourself. The format string passed in the original logging call. Merged with args to produce <code>message</code> , or an arbitrary object (see Using arbitrary objects as messages).
name	%(name)s	Name of the logger used to log the call.
pathname	%(pathname)s	Full pathname of the source file where the logging call was issued (if available).
process	%(process)d	Process ID (if available).
process-Name	%(processName)s	Process name (if available).
relativeCreated	%(relativeCreated)d	Time in milliseconds when the <code>LogRecord</code> was created, relative to the time the logging module was loaded.
stack_info		You shouldn't need to format this yourself. Stack frame information (where available) from the bottom of the stack in the current thread, up to and including the stack frame of the logging call which resulted in the creation of this record.
thread	%(thread)d	Thread ID (if available).
thread-Name	%(threadName)s	Thread name (if available).
taskName	%(taskName)s	<code>asyncio.Task</code> name (if available).

```
#log 출력 형식
formatter = logging.Formatter("%(asctime)s - %(name)s - %(levelname)s - %(messages)s")

# log를 console에 출력
stream_handler = logging.StreamHandler()
stream_handler.setFormatter(formatter)
logger.addHandler(stream_handler)

#log를 파일에 출력
file_handler = logging.FileHandler("my.log")
file_handler.setFormatter(formatter)
logger.addHandler(file_handler)

for i in range(10):
    logger.info(f"{i}번째 방문입니다.")
```

로깅 방법

- 쉬운 도구: python logging 라이브러리

- StreamHandler(): 터미널 출력
- FileHandler(): 파일 출력
- logger.TYPE(): 로깅
 - TYPE 수준으로 기록
 - Info로 기록했으면, debug()를 통해 로깅한 것은 출력/기록 안됨.

Config Level	Event Level					
	TRACE	DEBUG	INFO	WARN	ERROR	FATAL
ALL	○	○	○	○	○	○
TRACE	○	○	○	○	○	○
DEBUG	X	○	○	○	○	○
INFO	X	X	○	○	○	○
WARN	X	X	X	○	○	○
ERROR	X	X	X	X	○	○
FATAL	X	X	X	X	X	○
OFF	X	X	X	X	X	X

```
import logging

# 로그 생성
logger = logging.getLogger()

# 로그의 출력 기준 설정
logger.setLevel(logging.INFO)

#log 출력 형식
formatter = logging.Formatter("%(asctime)s - %(name)s - %(levelname)s - %(messages)s")

# log를 console에 출력
stream_handler = logging.StreamHandler()
stream_handler.setFormatter(formatter)
logger.addHandler(stream_handler)

#log를 파일에 출력
file_handler = logging.FileHandler("my.log")
file_handler.setFormatter(formatter)
logger.addHandler(file_handler)

for i in range(10):
    logger.info(f"{i}번째 방문입니다.")
```

로깅 방법

- 쉬운 도구: python logging 라이브러리
 - 터미널 출력/파일 출력 로깅 수준 조정 가능

```
import logging

# 로거 세팅
logger = logging.getLogger("postprocessor")
logger.setLevel(logging.DEBUG)

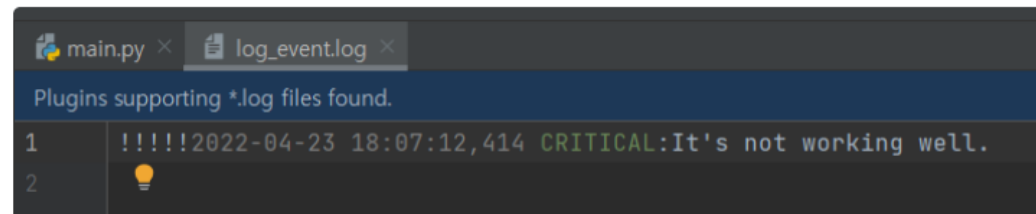
# 일반 핸들러, 포맷터 세팅
formatter = logging.Formatter("%(asctime)s %(levelname)s: %(message)s")
handler = logging.StreamHandler()
handler.setFormatter(formatter)

# 크리티컬 이벤트에 대한 핸들러, 포맷터 세팅
formatter_critical = logging.Formatter("!!!!!!%(asctime)s %(levelname)s: %(message)s")
handler_critical = logging.FileHandler("log_event.log")
handler_critical.setLevel(logging.CRITICAL)
handler_critical.setFormatter(formatter_critical)

# 각 핸들러를 로거에 추가
logger.addHandler(handler)
logger.addHandler(handler_critical)

logger.info("Is working well?")
logger.critical("It's not working well.")
```

```
2022-04-23 18:07:12,414 INFO:Is working well?
2022-04-23 18:07:12,414 CRITICAL:It's not working well.
```



예시 – INFO Logger

```
# main.py
import logging
from auth import login
from payment import pay

logger = logging.getLogger()
logger.setLevel(logging.INFO)
handler = logging.FileHandler("multi_module.log", encoding="utf-8")
formatter = logging.Formatter('%(asctime)s [%(levelname)s] [%(name)s] - %(message)s')
handler.setFormatter(formatter)
logger.addHandler(handler)

logger.debug("로그 설정 완료")
logger.info("시작")
login("admin", "wrong")
if login("admin", "1234"):
    pay("admin", 800)
    pay("admin", 8000)
logger.info("종료")
```

```
# auth.py
import logging
logger = logging.getLogger()

def login(user, pw):
    logger.debug(f"입력값 확인: user={user}, pw={pw}")
    logger.info(f"로그인 시도: {user}")

    if pw != "1234":
        logger.error("비밀번호 오류")
        return False

    logger.info("로그인 성공")
    return True
```

```
# payment.py
import logging
logger = logging.getLogger()

def pay(user, amt):
    logger.debug(f"결제 파라미터 확인: user={user}, amount={amt}")
    logger.info(f"결제 시도: {amt}")

    if amt > 1000:
        logger.warning("고액 결제")

    if amt > 5000:
        logger.critical("이상 결제")
        return False

    logger.info("결제 완료")
    return True
```

[2025-05-20 12:25:26,025]	[INFO]	[main]	] - 시작
[2025-05-20 12:25:26,025]	[INFO]	[auth]	] - 로그인 시도: admin
[2025-05-20 12:25:26,025]	[ERROR]	[auth]	] - 비밀번호 오류
[2025-05-20 12:25:26,025]	[INFO]	[auth]	] - 로그인 시도: admin
[2025-05-20 12:25:26,025]	[INFO]	[auth]	] - 로그인 성공
[2025-05-20 12:25:26,025]	[INFO]	[payment]	] - 결제 시도: 800
[2025-05-20 12:25:26,025]	[INFO]	[payment]	] - 결제 완료
[2025-05-20 12:25:26,025]	[INFO]	[payment]	] - 결제 시도: 8000
[2025-05-20 12:25:26,025]	[WARNING]	[payment]	] - 고액 결제
[2025-05-20 12:25:26,025]	[CRITICAL]	[payment]	] - 이상 결제
[2025-05-20 12:25:26,025]	[INFO]	[main]	] - 종료

예시 – INFO Logger

```
# main.py
import logging
from auth import login
from payment import pay

logger = logging.getLogger()
logger.setLevel(logging.DEBUG)
handler = logging.FileHandler("multi_module.log", encoding="utf-8")
formatter = logging.Formatter('%(asctime)s [%(levelname)s] [%(name)s] - %(message)s')
handler.setFormatter(formatter)
logger.addHandler(handler)

logger.debug("로그 설정 완료")
logger.info("시작")
login("admin", "wrong")
if login("admin", "1234"):
    pay("admin", 800)
    pay("admin", 8000)
logger.info("종료")
```

```
# auth.py
import logging
logger = logging.getLogger()

def login(user, pw):
    logger.debug(f"입력값 확인: user={user}, pw={pw}")
    logger.info(f"로그인 시도: {user}")

    if pw != "1234":
        logger.error("비밀번호 오류")
        return False

    logger.info("로그인 성공")
    return True
```

```
# payment.py
import logging
logger = logging.getLogger()

def pay(user, amt):
    logger.debug(f"결제 파라미터 확인: user={user}, amount={amt}")
    logger.info(f"결제 시도: {amt}")

    if amt > 1000:
        logger.warning("고액 결제")

    if amt > 5000:
        logger.critical("이상 결제")
        return False

    logger.info("결제 완료")
    return True
```

```
[2025-05-20 14:00:00,000] [DEBUG] [main] ] - 로깅 설정 완료
[2025-05-20 14:00:01,000] [INFO] [main] ] - 시작
[2025-05-20 14:00:02,000] [DEBUG] [auth] ] - 입력값 확인: user=admin, pw=wrong
[2025-05-20 14:00:03,000] [INFO] [auth] ] - 로그인 시도: admin
[2025-05-20 14:00:04,000] [ERROR] [auth] ] - 비밀번호 오류
[2025-05-20 14:00:05,000] [DEBUG] [auth] ] - 입력값 확인: user=admin, pw=1234
[2025-05-20 14:00:06,000] [INFO] [auth] ] - 로그인 시도: admin
[2025-05-20 14:00:07,000] [INFO] [auth] ] - 로그인 성공
[2025-05-20 14:00:08,000] [DEBUG] [payment] ] - 결제 파라미터 확인: user=admin, amount=800
[2025-05-20 14:00:09,000] [INFO] [payment] ] - 결제 시도: 800
[2025-05-20 14:00:10,000] [INFO] [payment] ] - 결제 완료
[2025-05-20 14:00:11,000] [DEBUG] [payment] ] - 결제 파라미터 확인: user=admin, amount=8000
[2025-05-20 14:00:12,000] [INFO] [payment] ] - 결제 시도: 8000
[2025-05-20 14:00:13,000] [WARNING] [payment] ] - 고액 결제
[2025-05-20 14:00:14,000] [CRITICAL] [payment] ] - 이상 결제
[2025-05-20 14:00:15,000] [INFO] [main] ] - 종료
```


실습 11. 로깅

운영 중 에러가 발생한다면?

- 의도적 에러 삽입 후 push

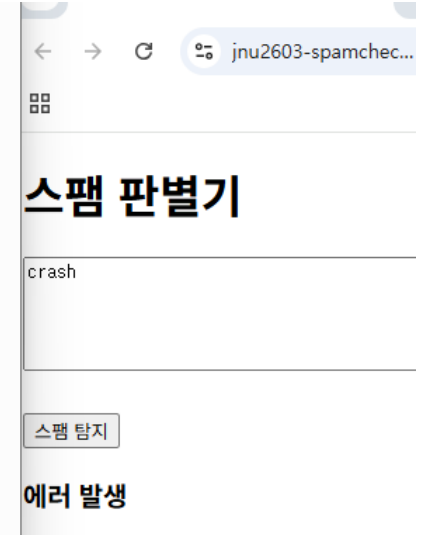
```
! docker-verify.yml X main.py X spam.py
2_initial SpamCheck_git > app > main.py > classify
22 @app.post("/classify")
23 async def classify(payload: ClassifyRequest):
24     text = payload.text
25     if text == "crash":
26         raise RuntimeError("의도적 장애 추가")
27     label, score = check_spam(text)
28     return {
29         "label": label, "score": score
30     }
```

```

03:15:41 PM ==> #####
03:17:56 PM [dlzjt] INFO: 10.23.85.196:0 - "GET / HTTP/1.1" 200 OK
03:18:26 PM [dlzjt] INFO: 10.19.235.3:0 - "POST /classify HTTP/1.1" 200 OK
03:18:31 PM [dlzjt] INFO: 10.19.191.3:0 - "GET / HTTP/1.1" 200 OK
03:18:58 PM [dlzjt] INFO: 10.19.191.3:0 - "POST /classify HTTP/1.1" 200 OK
03:19:24 PM [dlzjt] INFO: 10.23.85.196:0 - "POST /classify HTTP/1.1" 500 Internal Server Error
03:19:25 PM [dlzjt] ERROR: Exception in ASGI application

03:19:25 PM [dlzjt] Traceback (most recent call last):
03:19:25 PM [dlzjt] File "/usr/local/lib/python3.13/site-
packages/uvicorn/protocols/http/httptools_impl.py", line 416, in run_asgi
03:19:25 PM [dlzjt]     result = await app( # type: ignore[func-returns-value]
03:19:25 PM [dlzjt]         ~~~~~~
03:19:25 PM [dlzjt]     self.scope, self.receive, self.send
03:19:25 PM [dlzjt]     ~~~~~~
03:19:25 PM [dlzjt] )
03:19:25 PM [dlzjt] ^
03:19:25 PM [dlzjt] File "/usr/local/lib/python3.13/site-packages/uvicorn/middleware/proxy_headers.py",
line 60, in __call__
03:19:25 PM [dlzjt]     return await self.app(scope, receive, send)
03:19:25 PM [dlzjt]     ~~~~~~
03:19:25 PM [dlzjt] File "/usr/local/lib/python3.13/site-packages/fastapi/applications.py", line 1138,
03:19:25 PM [dlzjt] File "/usr/local/lib/python3.13/site-packages/fastapi/routing.py", line 725, in
run_endpoint_function
03:19:25 PM [dlzjt]     return await dependant.call(**values)
03:19:25 PM [dlzjt]     ~~~~~~
03:19:25 PM [dlzjt] File "/app/app/main.py", line 26, in classify
03:19:25 PM [dlzjt]     raise RuntimeError("외도적 장애 추가")
03:19:25 PM [dlzjt] RuntimeError: 외도적 장애 추가

```



- 실제로는 직관적인 에러보다 운영 중 “새로운 버그” 들이 보고됨 (유저 버그리포트, 개발자 회의 등)
- 500 에러는 보이지만..
- 어떤 요청? 누가? 얼마나 자주? 등 문제 해결에 필요한 실마리가 부족

그래서 로깅...

- Main에 logging 추가 후 push

```
INFO: Application startup complete.
INFO: 127.0.0.1:60284 - "GET / HTTP/1.1" 200 OK
2026-02-12 15:32:55,414 | INFO | main.py:38 (classify) | CALL /classify | text='crash' | len=5
2026-02-12 15:32:55,415 | ERROR | main.py:51 (classify) | FAIL /classify | text='crash' | error=RuntimeError: 의도적 장애 추가
Traceback (most recent call last):
  File "C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git\app\main.py", line 42, in classify
    raise RuntimeError("의도적 장애 추가")
RuntimeError: 의도적 장애 추가
INFO: 127.0.0.1:60284 - "POST /classify HTTP/1.1" 200 OK
```

스팸 판별기

crash

스팸 탐지

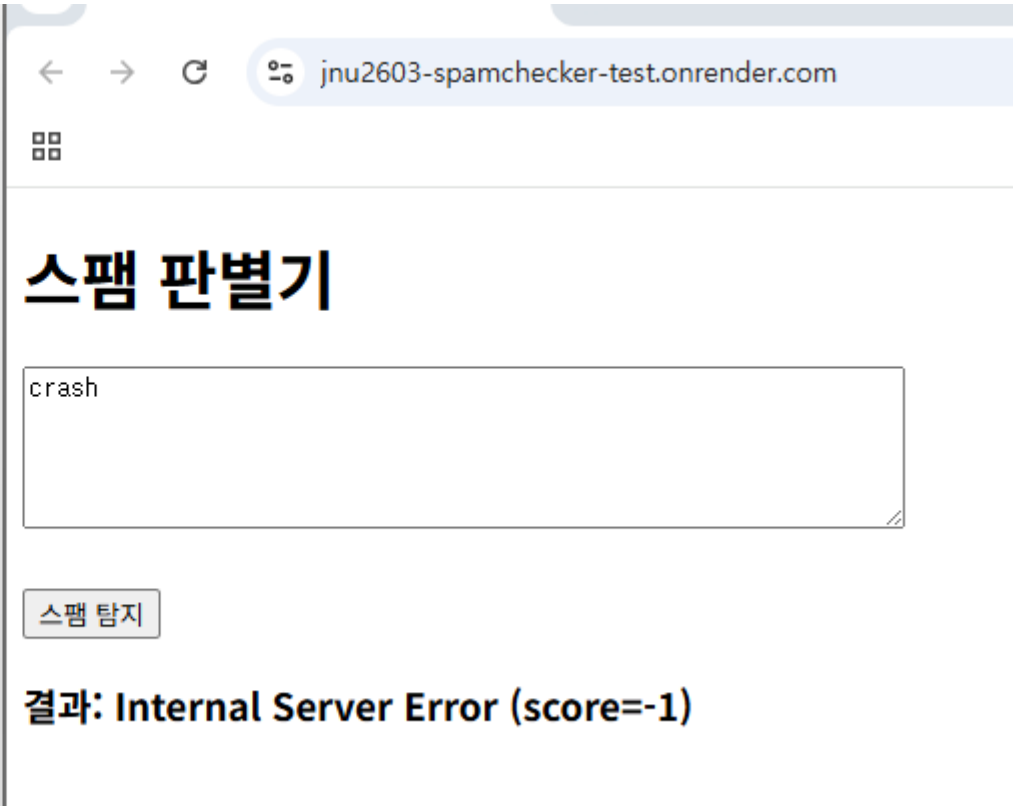
결과: Internal Server Error (score=-1)

```
# app/main.py
import logging
# 1) 로그 포맷: 시간 + 레벨 + 메시지
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | "
           "%(filename)s:%(lineno)d (%(funcName)s) | "
           "%(message)s"
)
logger = logging.getLogger("spamcheck")

@app.post("/classify")
async def classify(payload: ClassifyRequest):
    text = payload.text
    # (A) 요청 들어온 것 자체를 기록: 언제(로그시간) / 무엇(endpoint) / 어떤 입력
    logger.info(f"CALL /classify | text='{text}' | len={len(text)}")
    try:
        if text == "crash":
            raise RuntimeError("의도적 장애 추가")
        label, score = check_spam(text)
        # (B) 정상 처리 결과도 짧게 기록
        logger.info(f"OK /classify | label={label} score={score}")
    except Exception as e:
        # (C) 디버깅 핵심: 에러 종류/메시지 + 스택트레이스(파일/라인 포함)
        # logger.exception은 현재 예외의 traceback을 자동으로 찍어줍니다.
        logger.exception(
            f"FAIL /classify | text='{text}' | error={type(e).__name__}: {e}"
        )
        # (D) 사용자 응답은 심플하게
        return {"label": "Internal Server Error", "score": -1}
    return {
        "label": label, "score": score
    }
```

- 이제 에러가 뜨더라도, 어디에서 발생한 문제인지, 어떤 입력이 들어왔는지 바로 확인 가능 (디버깅 도움)

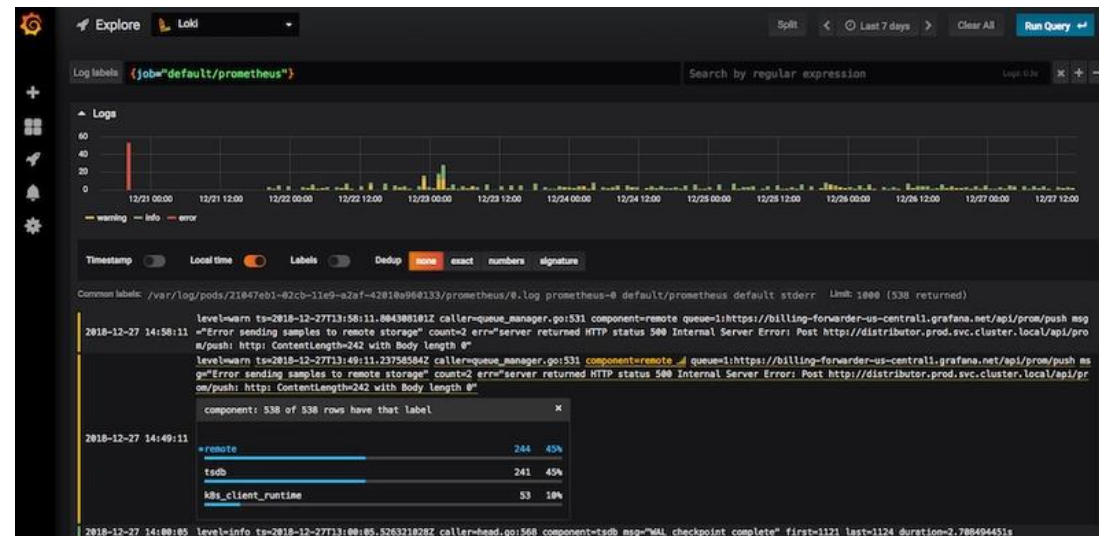
```
03:35:06 PM ==> Available at your primary URL https://jnu2603-spamchecker-test.onrender.com
03:35:06 PM ==>
03:35:06 PM ==> //////////////////////////////////////
03:36:35 PM [9n8xw] INFO: 10.19.191.3:0 - "GET / HTTP/1.1" 200 OK
03:36:46 PM [9n8xw] 2026-02-12 06:36:46,052 | INFO | main.py:38 (classify) | CALL /classify | text='hello
03:36:46 PM [9n8xw] ' | len=6
03:36:46 PM [9n8xw] 2026-02-12 06:36:46,052 | INFO | main.py:46 (classify) | OK /classify | label=ham
score=0
03:36:46 PM [9n8xw] INFO: 10.19.191.3:0 - "POST /classify HTTP/1.1" 200 OK
03:36:49 PM [9n8xw] 2026-02-12 06:36:49,618 | INFO | main.py:38 (classify) | CALL /classify | text='get
free prize ' | len=15
03:36:49 PM [9n8xw] 2026-02-12 06:36:49,619 | INFO | main.py:46 (classify) | OK /classify | label=spam
score=2
03:36:49 PM [9n8xw] INFO: 10.19.191.3:0 - "POST /classify HTTP/1.1" 200 OK
03:36:53 PM [9n8xw] 2026-02-12 06:36:53,065 | INFO | main.py:38 (classify) | CALL /classify |
text='crash' | len=5
03:36:53 PM [9n8xw] INFO: 10.19.235.3:0 - "POST /classify HTTP/1.1" 200 OK
03:36:53 PM [9n8xw] 2026-02-12 06:36:53,066 | ERROR | main.py:51 (classify) | FAIL /classify |
text='crash' | error=RuntimeError: 의도적 장애 추가
03:36:53 PM [9n8xw] Traceback (most recent call last):
03:36:53 PM [9n8xw] File "/app/app/main.py", line 42, in classify
03:36:53 PM [9n8xw] raise RuntimeError("의도적 장애 추가")
03:36:53 PM [9n8xw] RuntimeError: 의도적 장애 추가
```



근데 계속 로그를 지켜봐야 하나...

- 사실 플랫폼마다 이런 로그들을 시각적으로 잘 정리해서 보여주는 기능이 있고, 특정 에러 로그일 때 알람을 보내주는 기능, 시각화 기능 등이 제공됨
- 대규모 프로젝트에서는 수집되는 로그가 너무 많기 때문에, 이를 따로 관리하기도 함

- 수집 → 저장 → 검색 → 시각화 기능 제공
- 주요 툴 체인
 - ELK: Logstash (수집) → elastic search (저장/검색) → Kibana (시각화)
 - Grafana Loki



- 우리는 예시로 에러 발생 시 GitHub Issue를 자동 생성하는 것을 대상으로 함

- 그러나 실제로는 권고 안함. 에러가 계속 발생할 때 마다 이슈가 올라오면 너무 많기 때문.

준비

• Github token 발급

Settings / Developer Settings

GitHub Apps
OAuth Apps
Personal access tokens
Fine-grained tokens
Tokens (classic)

New fine-grained personal access token

Create a fine-grained, repository-scoped token suitable for personal AF

Token name *

spamcheck_test

✓ 'spamcheck_test' is available.
A unique name for this token. May be visible to resource owners or users with poss

Description

Resource owner

MisooKim

The token will only be able to make changes to resources owned by the selected re repositories.

Expiration

No expiration

⚠ GitHub strongly recommends that you set an expiration date for your token to

Repository access

- ☐ Public repositories
Read-only access to public repositories.
- ☐ All repositories
This applies to all current and future repositories you own. Also includes public repositories (read-only).
- ☒ Only select repositories
Select at least one repository. Max 50 repositories. Also includes public repositories (read-only).
- Select repositories 1
- Selected 1 repository.
- MisooKim/jnu2603-spamchecker_test

Permissions

Choose the minimal permissions necessary for your needs. [Learn more about permissions.](#)

Repositories 2 Account 0

+ Add permissions

Issues

Issues and related comments, assignees, labels, and milestones. [Learn more.](#)

Access: Read and write

Metadata **Required**

Search repositories, list collaborators, and access repository metadata. [Learn more.](#)

Access: Read-only

Generate token Cancel

This token will be ready for use immediately.

spamcheck_test

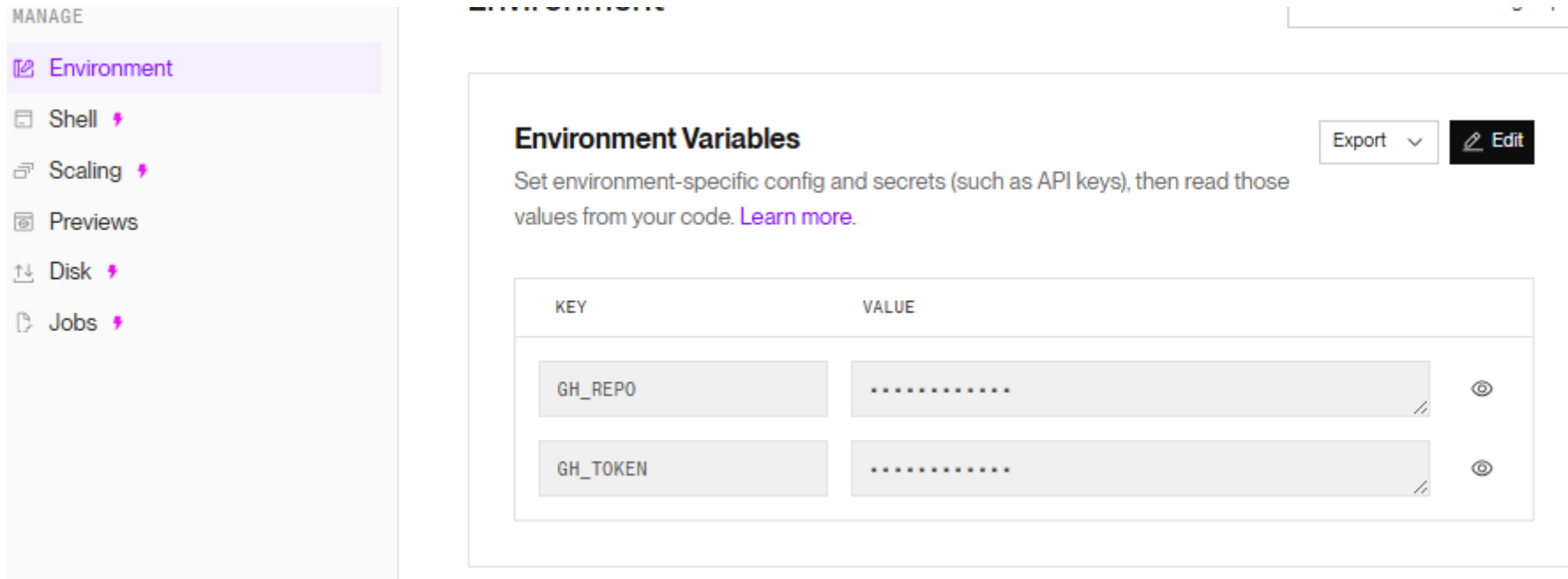
Never used • ⚠ This token has no expiration date

Make sure to copy your personal access token now as you will not be able to see this again.

github_pat_11AEZ7LAA0SCXLxsu0wtr_dZkdaHFY7P0DYVVUW8re8ZQsonowCoyCtdffxT7vo8aEPFT

준비

- Render에 환경 변수 지정
 - GH_REPO: MisooKim/jnu2603-spamchecker_test
 - GH_TOKEN: 아까 그 복사한 토큰



The screenshot shows the 'Environment Variables' configuration page in the Render dashboard. On the left, a sidebar under the 'MANAGE' tab lists 'Environment', 'Shell', 'Scaling', 'Previews', 'Disk', and 'Jobs'. The 'Environment' tab is selected. The main content area is titled 'Environment Variables' and includes an 'Export' dropdown and an 'Edit' button. Below this, a table lists the configured environment variables:

KEY	VALUE
GH_REPO	
GH_TOKEN	

코드 수정

• App/issue.py

```
import os
import logging
import requests
```

```
def create_github_issue(title: str, body: str, logger) -> None:
```

```
    repo = os.getenv("GH_REPO")
```

```
    token = os.getenv("GH_TOKEN")
```

```
    if not repo or not token:
```

```
        # 토큰 없으면 조용히 스킵(수업/실습에서 편함)
```

```
        logger.warning("GH_REPO/GH_TOKEN not set; skipping GitHub issue creation.")
```

```
        return
```

```
    url = f"https://api.github.com/repos/{repo}/issues"
```

```
    headers = {
```

```
        "Authorization": f"Bearer {token}",
```

```
        "Accept": "application/vnd.github+json",
```

```
    }
```

```
    payload = {"title": title, "body": body}
```

```
    r = requests.post(url, headers=headers, json=payload, timeout=10)
```

```
    if r.status_code >= 300:
```

```
        logger.warning(f"Failed to create issue: {r.status_code} {r.text[:200]}")
```

```
1 fastapi==0.128.0
2 uvicorn[standard]==0.40.0
3 python-multipart==0.0.21
4 pydantic==2.12.5
5 pytest==9.0.2
6 httpx==0.28.1
7 requests==2.32.5
```

• App/main.py

```
except Exception as e:
```

```
    # (C) 디버깅 핵심: 에러 종류/메시지 + 스택트레이스(파일/라인 포함)
```

```
    # logger.exception은 현재 예외의 traceback을 자동으로 찍어줍니다.
```

```
    logger.exception(
```

```
        f"FAIL /classify | text='{text}' | error={type(e).__name__}: {e}"
```

```
    )
```

```
# (D) GitHub Issue 자동 생성
```

```
tb = traceback.format_exc()
```

```
title = f"[Prod Error] /classify failed: {type(e).__name__}"
```

```
body = (
```

```
    f"## Summary\n"
```

```
    f"- endpoint: /classify\n"
```

```
    f"- input(text, short): `{text}`\n"
```

```
    f"- input length: {len(text)}\n\n"
```

```
    f"## Exception\n"
```

```
    f"- type: {type(e).__name__}\n"
```

```
    f"- message: {str(e)}\n\n"
```

```
    f"## Traceback (line info)\n"
```

```
    f"``text\n{tb}\n``"
```

```
)
```

```
create_github_issue(title, body, logger)
```

```
# (D) 사용자 응답은 심플하게
```

```
return {"label": "Internal Server Error", "score": -1}
```

main.py

```
2_initial SpamCheck_git > app > main.py > ...
6 from app.spam import check_spa
7 from pydantic import BaseModel
8 from app.issue import *
9 import logging
```


실행 배포

- 로컬에서
- Render에서 (환경 변수 설정 후 재부팅 필수)

```
INFO:      Application startup complete.
INFO:      127.0.0.1:53065 - "GET / HTTP/1.1" 200 OK
2026-02-12 15:43:51,266 | INFO | main.py:39 (classify) | CALL /classify | text='crash' | len=5
2026-02-12 15:43:51,267 | ERROR | main.py:52 (classify) | FAIL /classify | text='crash' | error=RuntimeError: 의도적 장애 추가
Traceback (most recent call last):
  File "C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git\app\main.py", line 43, in classify
    raise RuntimeError("의도적 장애 추가")
RuntimeError: 의도적 장애 추가
2026-02-12 15:43:51,267 | WARNING | issue.py:10 (create_github_issue) | GH_REPO/GH_TOKEN not set; skipping GitHub issue creation.
INFO:      127.0.0.1:53065 - "POST /classify HTTP/1.1" 200 OK
```

☐ Open 2 Closed 2 Author ▾

☐ [Prod Error] /classify failed: RuntimeError
#7 · MisookKim opened now

☐ [Prod Error] /classify failed: RuntimeError
#6 · MisookKim opened now

[Prod Error] /classify failed: RuntimeError #7

Open



MisookKim opened now

Summary

- endpoint: /classify
- input(text, short): crash
- input length: 5

Exception

- type: RuntimeError
- message: 의도적 장애 추가

Traceback (line info)

```
Traceback (most recent call last):
  File "/app/app/main.py", line 43, in classify
    raise RuntimeError("의도적 장애 추가")
RuntimeError: 의도적 장애 추가
```



이슈 해결

- git switch -c fix/issue-7-runtime-error
- 코드 수정 → git add . → git commit -m "fix issue #7"
→ git switch main → git merge fix/issue-7-runtime-error → git push main

```
main.py • spam.py
2_initial SpamCheck_git > app > main.py > classify
34 async def classify(payload: ClassifyRequest):
39
40     try:
41         if text == "crash":
42             raise RuntimeError("의도적 장애 추가")
43         label, score = check_spam(text)
44
```

[Prod Error] /classify failed: RuntimeError #6

Closed



MisookKim opened 7 minutes ago

Summary

- endpoint: /classify
- input(text, short): crash
- input length: 5

Exception

- type: RuntimeError
- message: 의도적 장애 추가

Traceback (line info)

Traceback (most recent call last):
File "/app/app/main.py", line 43, in classify
raise RuntimeError("의도적 장애 추가")
RuntimeError: 의도적 장애 추가

Create sub-issue



MisookKim now

Duplicated #7



MisookKim closed this as completed now

[Prod Error] /classify failed: RuntimeError #7

Closed



MisookKim opened 7 minutes ago

Summary

- endpoint: /classify
- input(text, short): crash
- input length: 5

Exception

- type: RuntimeError
- message: 의도적 장애 추가

Traceback (line info)

Traceback (most recent call last):
File "/app/app/main.py", line 43, in classify
raise RuntimeError("의도적 장애 추가")
RuntimeError: 의도적 장애 추가

Create sub-issue

MisookKim added a commit that references this issue 2 minutes ago

fix issue #7

MisookKim mentioned this 1 minute ago

[Prod Error] /classify failed: RuntimeError #6



MisookKim now

Closed by c573410



MisookKim closed this as completed now

WEB SERVICE

jnu2603-spamchecker_test Docker

Service ID: srv-d66mac6sb7us73bfurog

MisookKim / jnu2603-spamchecker_test main

<https://jnu2603-spamchecker-test.onrender.com>

ⓘ Your free instance will spin down with inactivity, which can

Filter events 31

✓ Deploy live for c573410: fix issue #7

February 12, 2026 at 4:05 PM

Deploy started for c573410: fix issue #7

New commit via Auto-Deploy

February 12, 2026 at 4:02 PM

스팸 판별기

crash

스팸 탐지

결과: ham (score=0)

숙제 7. 실습 11

- 이번 주 일요일 자정까지
- 자유 양식

1. 실습 11

- 목적, 방법, 결과 스크린샷 및 설명
- 나중에 GitHub와 연동하여 이력 확인할 것이니 매주 복습 잘하기